

BÁO CÁO BÀI TẬP NHÓM

Đề bài: Retrieval-Augmented Generation System for Pittsburgh and CMU
Question Answering

Môn học: Ứng dụng trí tuệ nhân tạo cho ngôn ngữ

Thành viên nhóm

1. Nguyễn Thế Cường - 23020337
2. Ngô Nguyễn Khải Hưng - 23020382
3. Vũ Bảo Chinh - 23020341
4. Lê Hồng Anh - 23020327

Ngày 28 tháng 5 năm 2026

1. Giới thiệu

Bài tập này tập trung vào xây dựng một hệ thống hỏi đáp factual question answering về Pittsburgh và Carnegie Mellon University (CMU). Các câu hỏi trong bài có thể liên quan đến lịch sử, văn hóa, sự kiện, địa điểm, thể thao, ẩm thực và các thông tin đặc thù của CMU. Do nhiều thông tin thuộc miền hẹp hoặc có tính cập nhật, việc chỉ sử dụng mô hình ngôn ngữ lớn theo kiểu closed-book có thể dẫn đến câu trả lời thiếu chính xác hoặc không có căn cứ.

Để giải quyết vấn đề này, nhóm xây dựng một hệ thống Retrieval-Augmented Generation (RAG). Hệ thống trước hết thu thập và tiền xử lý các tài liệu công khai liên quan đến Pittsburgh và CMU, sau đó chia tài liệu thành các đoạn nhỏ để lập chỉ mục truy hồi. Khi nhận một câu hỏi, hệ thống truy xuất các đoạn tài liệu liên quan nhất và đưa chúng vào prompt cho mô hình ngôn ngữ để sinh câu trả lời ngắn gọn.

Trong báo cáo này, nhóm trình bày toàn bộ quy trình phát triển hệ thống, bao gồm tạo kho tri thức, tiền xử lý dữ liệu, chú thích tập train/test, xây dựng các biến thể mô hình và đánh giá kết quả. Ba cấu hình chính được so sánh gồm baseline closed-book, RAG với dense retrieval, và RAG có bổ sung reranker. Mục tiêu cuối cùng là tạo ra một pipeline có thể sinh một câu trả lời trên mỗi dòng, đúng định dạng yêu cầu và có hiệu quả tốt hơn so với mô hình không sử dụng truy hồi tài liệu.

2. Tạo dữ liệu

2.1. Biên soạn kho tri thức

Nhóm xây dựng kho tri thức từ các nguồn công khai liên quan đến Pittsburgh và CMU. Các nguồn chính bao gồm Wikipedia, Britannica, trang thông tin thành phố Pittsburgh, Visit Pittsburgh, các trang về lịch sử CMU, bảo tàng, sự kiện văn hóa, lễ hội ẩm thực, thể thao và một số trang sự kiện địa phương. Khi chọn tài liệu, nhóm ưu tiên các nguồn có nội dung factual, có thể dùng để trả lời các câu hỏi ngắn về lịch sử, địa điểm, tổ chức, sự kiện định kỳ và các thông tin đặc trưng của Pittsburgh/CMU.

Mỗi tài liệu được lưu kèm metadata gồm URL gốc, chủ đề, loại tài liệu, ngày thu thập và trạng thái tải xuống. Việc lưu metadata giúp hệ thống có thể truy vết nguồn tài liệu khi phân tích lỗi hoặc kiểm tra lại câu trả lời.

2.2. Thu thập và xử lý dữ liệu thô

Dữ liệu thô được thu thập bằng script crawl/download tự viết. Các trang HTML được tải về và lưu trong thư mục `data/raw/html/`; các tài liệu PDF được lưu trong `data/raw/pdf/`. Metadata của quá trình thu thập được lưu trong `data/raw/metadata/metadata.csv`. Tổng cộng nhóm thử thu thập 87 nguồn, trong đó 64 nguồn tải xuống thành công. Một số nguồn

không tải được do giới hạn truy cập như lỗi 403 hoặc 406.

Sau khi thu thập, nhóm tiền xử lý dữ liệu để tạo corpus sạch cho retrieval. Với HTML, nhóm sử dụng BeautifulSoup để loại bỏ script, style, menu, footer và các phần boilerplate không cần thiết. Với PDF, nhóm sử dụng pypdf và pdfplumber để trích xuất văn bản. Sau đó văn bản được chuẩn hóa khoảng trắng, loại bỏ dòng nhiều và chia thành các chunk ngắn để phục vụ truy hồi.

Kết quả tiền xử lý gồm 62 tài liệu sạch và 774 chunks. Mỗi chunk được gán với một `chunk_id`, `source_id`, URL nguồn, tiêu đề tài liệu và nội dung văn bản. Corpus cuối cùng được lưu trong `data/processed/corpus.jsonl`, còn các chunk phục vụ truy hồi được lưu trong `data/chunks/chunks.jsonl` và `data/chunks/chunks.csv`.

2.3. Tạo tập train và test

Nhóm tự tạo bộ câu hỏi–trả lời dựa trên kho tri thức đã thu thập. Các câu hỏi được viết bằng tiếng Anh để phù hợp với định dạng bài toán. Câu trả lời được chuẩn hóa theo hướng ngắn gọn, thường chỉ gồm một cụm danh từ, tên riêng, năm hoặc địa điểm. Với những câu hỏi có nhiều đáp án đúng, các đáp án được phân tách bằng dấu chấm phẩy.

Bộ dữ liệu cuối cùng gồm 120 cặp question-answer, được chia thành 80 mẫu train và 40 mẫu test. Tập train được dùng để phát triển prompt và kiểm tra pipeline trong quá trình xây dựng hệ thống. Tập test được giữ cố định để đánh giá các biến thể mô hình.

Tập dữ liệu	Số câu hỏi	Mục đích
Train	80	Phát triển hệ thống và kiểm tra prompt
Test	40	Đánh giá nội bộ các mô hình

Bảng 1: Thống kê tập dữ liệu question-answer được chú thích.

2.4. Đánh giá chất lượng chú thích

Để kiểm tra chất lượng dữ liệu chú thích, nhóm lấy ngẫu nhiên 25 câu hỏi từ tập test và chuẩn bị hai file chú thích độc lập cho hai người chú thích. Mỗi người điền câu trả lời vào cột `annotated_answer` dựa trên câu hỏi và nguồn tài liệu liên quan. Sau đó nhóm tính độ đồng thuận giữa hai bộ chú thích bằng exact agreement và token-level F1.

Kết quả cho thấy exact agreement đạt 1.00 và token-level F1 đạt 1.00 trên 25 mẫu được kiểm tra. Điều này cho thấy các câu hỏi trong tập test có đáp án rõ ràng và cách chuẩn hóa câu trả lời giữa hai người chú thích là nhất quán.

3. Mô hình và phương pháp

3.1. Baseline closed-book

Baseline đầu tiên là mô hình closed-book, tức là mô hình ngôn ngữ trả lời trực tiếp câu hỏi mà không được cung cấp tài liệu truy hồi. Nhóm sử dụng mô hình Qwen2.5-3B-Instruct, chạy trên Google Colab và triển khai dưới dạng API thông qua ngrok. Với mỗi câu hỏi, hệ thống gửi trực tiếp prompt đến API và yêu cầu mô hình sinh một câu trả lời ngắn.

Baseline này được dùng để kiểm tra khả năng trả lời dựa trên kiến thức sẵn có của mô hình. Đây cũng là mốc so sánh chính để đánh giá xem retrieval có thực sự cải thiện kết quả hay không.

3.2. RAG với dense retrieval

Biến thể chính của hệ thống là Retrieval-Augmented Generation với dense retrieval. Pipeline gồm ba bước: nhúng câu hỏi, truy hồi tài liệu liên quan và sinh câu trả lời dựa trên context truy hồi được.

Đầu tiên, toàn bộ các chunk trong kho tri thức được mã hóa bằng mô hình embedding `sentence-transformers/all-MiniLM-L6-v2`. Mô hình này tạo vector embedding 384 chiều cho mỗi chunk. Các embedding sau đó được lưu vào Qdrant, một vector database hỗ trợ tìm kiếm tương đồng nhanh bằng cosine similarity.

Khi nhận một câu hỏi, hệ thống mã hóa câu hỏi bằng cùng mô hình embedding, sau đó truy hồi top-5 chunks có độ tương đồng cao nhất từ Qdrant. Các chunks này được ghép vào prompt dưới dạng context. Mô hình Qwen2.5-3B-Instruct sau đó sinh câu trả lời dựa trên context được cung cấp. Prompt được thiết kế để yêu cầu mô hình trả lời ngắn gọn, không giải thích dài và trả về unknown nếu không tìm thấy thông tin trong context.

3.3. RAG với reranker

Biến thể thứ ba bổ sung một bước reranking sau dense retrieval. Thay vì chỉ lấy trực tiếp top-5 chunks từ Qdrant, hệ thống trước tiên truy hồi top-20 chunks bằng dense retrieval. Sau đó, nhóm sử dụng mô hình cross-encoder `cross-encoder/ms-marco-MiniLM-L-6-v2` để chấm điểm lại mức độ liên quan giữa câu hỏi và từng chunk.

Cross-encoder nhận trực tiếp cặp `question`, `chunk` làm đầu vào, do đó có khả năng đánh giá quan hệ ngữ nghĩa chi tiết hơn so với việc chỉ so sánh hai vector embedding độc lập. Sau khi rerank, hệ thống chọn top-5 chunks có điểm cao nhất để đưa vào prompt cho mô hình sinh câu trả lời.

3.4. Lý do lựa chọn phương pháp

Nhóm chọn all-MiniLM-L6-v2 vì mô hình nhỏ, chạy được cục bộ, tốc độ nhanh và phù hợp với bài toán retrieval trên tập tài liệu vừa phải. Qdrant được chọn vì hỗ trợ vector search đơn giản, có thể chạy local và dễ tích hợp với Python. Mô hình Qwen2.5-3B-Instruct được chọn làm reader/generator vì đây là mô hình instruction-tuned mã nguồn mở, có thể truy cập qua HuggingFace và phù hợp với chính sách mô hình của bài tập.

Biến thể reranker được thêm vào vì dense retrieval đôi khi lấy về các chunk có từ khóa giống câu hỏi nhưng không chứa đáp án chính xác. Reranker giúp sắp xếp lại các chunk theo mức độ liên quan thực tế trước khi đưa vào mô hình ngôn ngữ, từ đó kỳ vọng cải thiện độ chính xác của câu trả lời.

4. Thực nghiệm và kết quả

4.1. Thiết lập thực nghiệm

Nhóm đánh giá ba hệ thống trên tập test nội bộ gồm 40 câu hỏi. Tất cả các hệ thống đều sinh đầu ra theo định dạng một dòng trả lời cho mỗi câu hỏi. Các câu trả lời được đánh giá bằng ba chỉ số theo phong cách SQuAD: exact match, token-level F1 và answer recall.

Exact match đo tỉ lệ câu trả lời khớp hoàn toàn với đáp án tham chiếu sau khi chuẩn hóa. Token-level F1 đo mức độ trùng lặp token giữa câu trả lời dự đoán và đáp án đúng. Answer recall đo tỉ lệ token trong đáp án tham chiếu được bao phủ bởi câu trả lời của hệ thống.

Ba hệ thống được so sánh gồm:

- **Closed-book:** Qwen2.5-3B-Instruct trả lời trực tiếp không dùng tài liệu truy hồi.
- **RAG top-5:** truy hồi top-5 chunks từ Qdrant bằng dense retrieval.
- **RAG rerank:** truy hồi top-20 chunks, sau đó rerank và chọn top-5 chunks tốt nhất.

4.2. Kết quả tổng thể

Bảng 2 trình bày kết quả tổng thể trên tập test nội bộ. Kết quả cho thấy cả hai hệ thống RAG đều vượt rõ rệt baseline closed-book. Trong đó, RAG có reranker đạt kết quả cao nhất với exact match 0.8000 và token-level F1 0.8618.

Hệ thống	Số câu hỏi	EM	F1	Recall
Closed-book	40	0.1500	0.1892	0.2083
RAG top-5	40	0.6750	0.7690	0.7662
RAG rerank	40	0.8000	0.8618	0.8525

Bảng 2: Kết quả tổng thể trên tập test nội bộ.

So với closed-book, RAG top-5 tăng exact match từ 0.1500 lên 0.6750. Khi thêm reranker, exact match tiếp tục tăng lên 0.8000. Điều này cho thấy việc cung cấp context từ kho tri thức là yếu tố quan trọng nhất, trong khi reranker giúp cải thiện thêm chất lượng các đoạn tài liệu được đưa vào prompt.

4.3. Kết quả theo nhóm câu hỏi

Nhóm cũng phân tích kết quả theo từng loại câu hỏi. Bảng 3 báo cáo exact match theo topic. Các nhóm câu hỏi về CMU, lịch sử CMU và văn hóa/âm nhạc đạt kết quả rất tốt khi dùng RAG. Các nhóm khó hơn như sports, food events và museums có kết quả thấp hơn do một số câu hỏi yêu cầu thông tin cụ thể hơn hoặc phụ thuộc vào việc truy hồi đúng chunk.

Topic	Closed-book	RAG top-5	RAG rerank
CMU	0.0000	1.0000	1.0000
CMU history	0.0000	0.6667	1.0000
Culture/music	0.5000	1.0000	1.0000
Food events	0.0000	0.7500	0.7500
General history	0.2500	0.5000	0.7500
Museums	0.0000	0.7500	0.7500
Museums history	1.0000	0.0000	1.0000
Sports	0.2000	0.6000	0.7000

Bảng 3: Exact match theo từng nhóm câu hỏi.

Kết quả theo topic cho thấy reranker cải thiện rõ nhất ở các nhóm *CMU history*, *general history*, *museums history* và *sports*. Điều này phù hợp với kỳ vọng vì các nhóm này thường có nhiều tài liệu gần giống nhau về mặt từ khóa, khiến dense retrieval dễ lấy nhầm chunk nếu không có bước xếp hạng lại.

Chúng ta có thể thấy khoảng cách giữa closed-book và các hệ thống RAG là lớn, cho thấy retrieval có tác động rõ ràng đến chất lượng trả lời trong thiết lập thực nghiệm này.

5. Phân tích

5.1. So sánh RAG và closed-book

Kết quả thực nghiệm cho thấy closed-book hoạt động kém hơn đáng kể so với các hệ thống RAG. Baseline closed-book chỉ đạt 0.1500 exact match và 0.1892 token-level F1. Nguyên nhân chính là mô hình phải dựa hoàn toàn vào kiến thức tham số đã học trước, trong khi nhiều câu hỏi trong bài yêu cầu thông tin cụ thể về Pittsburgh, CMU, sự kiện địa phương hoặc các địa danh không phổ biến.

Ngược lại, RAG top-5 đạt 0.6750 exact match và 0.7690 token-level F1. Việc bổ sung context từ kho tri thức giúp mô hình tìm được câu trả lời trong tài liệu thay vì phải suy đoán. Điều này đặc biệt rõ ở các nhóm câu hỏi như CMU, food events, museums và sports, nơi closed-book thường trả lời sai hoặc không đủ cụ thể.

5.2. Ảnh hưởng của reranker

Biến thể RAG có reranker đạt kết quả tốt nhất với 0.8000 exact match và 0.8618 token-level F1. So với RAG top-5, reranker cải thiện exact match thêm 0.1250 điểm. Cải thiện này đến từ việc reranker đánh giá lại mức độ liên quan giữa câu hỏi và từng chunk sau bước dense retrieval.

Dense retrieval dùng embedding độc lập cho câu hỏi và tài liệu, vì vậy đôi khi lấy về các chunk có từ khóa giống nhau nhưng không trực tiếp chứa đáp án. Cross-encoder reranker xử lý đồng thời cặp câu hỏi–chunk, nên có khả năng chọn context chính xác hơn. Hiệu quả này thể hiện rõ ở các nhóm câu hỏi như *CMU history*, *general history*, *museums history* và *sports*, nơi RAG rerank đạt exact match cao hơn RAG top-5.

5.3. Phân tích lỗi

Một số lỗi còn lại đến từ ba nguyên nhân chính. Thứ nhất, hệ thống đôi khi trả lời đúng về mặt ngữ nghĩa nhưng khác định dạng với đáp án tham chiếu, ví dụ dùng chữ thay vì số hoặc thêm từ phụ không cần thiết. Những trường hợp này có thể làm exact match thấp hơn dù token-level F1 vẫn tương đối cao.

Thứ hai, có những câu hỏi yêu cầu thông tin rất cụ thể, nhưng chunk được truy hồi chưa chứa đúng câu trả lời. Khi context thiếu thông tin, mô hình có xu hướng trả lời unknown hoặc sinh câu trả lời dựa trên thông tin gần đúng. Đây là hạn chế trực tiếp của bước thu thập dữ liệu và truy hồi tài liệu.

Thứ ba, một số lỗi đến từ việc mô hình sinh câu trả lời dài hơn yêu cầu. Vì metric của bài toán ưu tiên câu trả lời ngắn, việc thêm giải thích hoặc ngữ cảnh phụ có thể làm giảm exact match và F1. Do đó, prompt được thiết kế để yêu cầu mô hình chỉ trả về một cụm trả lời ngắn.

5.4. Ví dụ đầu ra

Bảng 4 trình bày một số ví dụ so sánh đầu ra giữa các hệ thống. Các ví dụ này cho thấy RAG thường sửa được lỗi của closed-book bằng cách sử dụng context truy hồi, trong khi reranker tiếp tục cải thiện khi dense retrieval lấy về context chưa tối ưu.

Câu hỏi	Closed-book	RAG top-5	RAG rerank
Which two rivers meet to form the Ohio River at Pittsburgh?	Allegheny and Ohio	Allegheny and Monongahela	Allegheny and Monongahela
What nickname refers to Pittsburgh's historical steel industry?	unknown	Steel City	Steel City
What year was Carnegie Mellon University founded?	1900	1900	1900
What MLB team plays in Pittsburgh?	Pittsburgh Pirates	Pittsburgh Pirates	Pittsburgh Pirates

Bảng 4: Ví dụ đầu ra của các hệ thống.

Nhìn chung, phân tích định lượng và định tính đều cho thấy chiến lược retrieve-and-augment là cần thiết cho bài toán này. Reranker giúp cải thiện thêm hiệu quả, đặc biệt ở các câu hỏi có nhiều tài liệu liên quan gần giống nhau.

6. Kết luận

Trong bài tập này, nhóm đã xây dựng một hệ thống RAG hoàn chỉnh cho bài toán hỏi đáp về Pittsburgh và CMU. Hệ thống bao gồm các bước thu thập dữ liệu công khai, tiền xử lý văn bản, chia chunk, lập chỉ mục bằng Qdrant, truy hồi bằng embedding model và sinh câu trả lời bằng Qwen2.5-3B-Instruct.

Kết quả thực nghiệm cho thấy RAG cải thiện rõ rệt so với baseline closed-book. Biểu thể tốt nhất là RAG có reranker, đạt 0.8000 exact match và 0.8618 token-level F1 trên tập test nội bộ. Điều này cho thấy việc truy hồi tài liệu liên quan trước khi sinh câu trả lời là cần thiết đối với các câu hỏi thuộc miền hẹp và có nhiều thông tin cụ thể.

Hạn chế chính của hệ thống là chất lượng còn phụ thuộc vào độ bao phủ của kho tri thức và độ chính xác của bước truy hồi. Trong tương lai, hệ thống có thể được cải thiện bằng cách mở rộng nguồn dữ liệu, làm sạch corpus tốt hơn và thử thêm các reranker hoặc mô hình reader mạnh hơn.